Deep technical and mathematical audit of pid-rs

Direct repository, source, proof-boundary, API, and publication review

OpenAI GPT-5.6 Pro

2026-08-12

Contents

1 Executive verdict **3**
1.1 Highest-priority conclusions . . . 3

2 Scope, evidence, and limitations **4**
2.1 Repository identity audited . . . 4
2.2 What I directly inspected . . . 4
2.3 Important limitation: no false claim of complete byte review . . . 4
2.4 Evidence grades used here . . . 4

3 Prioritized findings **5**

4 Detailed findings and required fixes **6**
4.1 RB-01 - The release's "reviewed source" claim is contradicted by its own ledger . . . 6
4.1.1 Evidence . . . 6
4.1.2 Why it matters . . . 6
4.1.3 Required patch . . . 6
4.1.4 Acceptance test . . . 6
4.2 RB-02 - Scope binding is fragmented across tag, current main, claims, and generated artifacts 6
4.2.1 Required patch: `AUDIT_STATE.json` . . . 7
4.3 RB-03 - The correspondence ladder is open even where formal and numerical assurance is strong 7
4.3.1 The five-edge ladder . . . 7
4.3.2 Required patch: assurance graph . . . 7
4.4 H-01 - Source errata need to become executable, first-class repository data . . . 8
4.5 H-02 - Python legacy flattening discards provenance that the Rust design treats as safety-critical 8
4.5.1 Required patch . . . 8
4.5.2 Acceptance test . . . 8
4.6 H-03 - The workflow mixes PID assurance with unrelated process narratives . . . 9
4.6.1 Required split . . . 9
4.7 H-04 - PDF publication contracts are not uniform or independently inspectable . . . 9
4.7.1 Required PDF contract . . . 9
4.8 H-05 - Implementation separation is not the same as independent review . . . 9
4.9 H-06 - The continuous-paper reference code is not a sufficient durable source pin . . . 10
4.10 M-01 and M-02 - Generated method catalog is complete but not human-reviewable . . . 10
4.10.1 Required presentation . . . 10
4.11 M-03 - Unit safety should be structural, not lexical . . . 11
4.12 M-04 - Continuous PID2 requires joint uncertainty . . . 11
4.13 M-05 - Numerical methods need diagnostic and oracle depth proportional to assurance claims 11
4.14 M-06 and M-07 - Dependency and CI governance . . . 11
4.15 M-08 - Define "independent" every time . . . 12
4.16 M-09 - Narrow the Cantor-law objection . . . 12
4.17 M-10 - Keep recent impossibility work in the right evidentiary category . . . 12
4.18 M-11 - Quantizer semantics need explicit versioned names . . . 12
4.19 M-12 - Replace narrative assurance with a queryable claim graph . . . 13

5 Mathematical source audit **13**

5.1 Ehrlich et al. continuous shared exclusions . . . 13
5.1.1 Equation (8): repeated source-bin factor . . . 13
5.1.2 Post-Definition-2 integral: repeated differential . . . 13
5.1.3 Equation (14): unit mismatch . . . 13
5.1.4 Algorithm 6: omitted antichain and wrong target count routine . . . 13
5.1.5 What remains unproved . . . 13
5.2 Schick-Poland et al. measure-theoretic shared exclusions . . . 13
5.2.1 Missing conditional normalization . . . 14
5.2.2 Bimeasurable is not bicontinuous . . . 14
5.2.3 RCP values at null conditioning events . . . 14
5.2.4 Borel isomorphism and the Cantor nuance . . . 14
5.2.5 Implementation judgment . . . 14
5.3 Makkeh-Gutknecht-Wibral categorical SxPID . . . 14
5.4 Williams-Beer `I_min` . . . 14
5.5 Kraskov-Stoegbauer-Grassberger mutual information . . . 15
5.6 Newer source routing . . . 15
5.6.1 Shannon invariants . . . 15
5.6.2 Barà mixed-variable estimator . . . 15
5.6.3 Multivariate inconsistencies and structural impossibility . . . 15

6 Formal methods and proof-boundary audit 15
6.1 What is genuinely strong . . . 15
6.2 What remains open . . . 16
6.3 Naming rules . . . 16
6.4 Proof adequacy tests . . . 16

7 PDF and Markdown workflow audit 16
7.1 Mathematical workflow document . . . 16
7.2 `METHODS.md` . . . 17
7.3 Generated PDF set . . . 17

8 Module-by-module code review matrix 17

9 Cross-cutting API and scientific-contract improvements 20
9.1 Separate numeric values from evidence and interpretation . . . 20
9.2 Treat negative capability as a first-class feature . . . 20
9.3 Make same-sample descriptive routes impossible to confuse with estimators . . . 20
9.4 Add cancellation/conditioning diagnostics for signed atoms . . . 20

10 Testing strategy: what to retain and what to add 21
10.1 Retain . . . 21
10.2 Add . . . 21

11 CI, release, and supply-chain review 21
11.1 What is correct . . . 21
11.2 What should change before 1.0 . . . 21

12 What the authors got right 22

13 A pre-1.0 remediation sequence 22
13.1 0.9.1 - Truth and scope . . . 22
13.2 0.9.2 - Scientific correspondence . . . 22
13.3 0.9.3 - API and numerics . . . 22
13.4 1.0 - Reviewable release . . . 22

14 Concrete acceptance checklist 23

15 Appendix A - v0.9.0 inventory metrics 23

16 Appendix B - External source inventory from the mathematical workflow 24

17 Appendix C - External source inventory from `METHODS.md` 27

18 Appendix D - Key repository artifacts reviewed **29**

19 Appendix E - Audit limitations and confidence boundaries **29**

20 Final assessment **29**

1 Executive verdict

`pid-rs` is substantially better engineered than the average research-code repository. Its safe-Rust core, explicit unsupported routes, finite-input checks, resource preflights, strict array-layout handling, careful categorical SxPID semantics, and unusually candid formal-proof boundaries are real strengths. The repository also catches several genuine defects and ambiguities in its primary scientific sources that many implementations would silently inherit.

The main problem is not ordinary memory unsafety or a single catastrophic formula bug. The main problem is **epistemic labeling and scope control**: the presentation sometimes runs ahead of what has actually been independently reviewed or linked end to end. In particular, the v0.9.0 release calls its contents "reviewed source," while its own 186-row file ledger says every file is `UNASSIGNED` and `INVENTORIED_NOT_REVIEWED`. The current `main` branch is also much larger than the released tag, while generated papers, exact-real proofs, high-precision checkers, Rust tests, and scientific-validity prose sit close enough together that a reader can overgeneralize one evidence class into another.

My release judgment is therefore:

> **Do not make a 1.0 assurance/certification claim until the review-state contradiction is corrected, every public claim is bound to an exact commit and artifact, and the paper-to-specification-to-formal-model-to-Rust correspondence gaps are represented as first-class open edges rather than buried caveats.**

This is not a recommendation to throw away the project. The core is promising and the authors' judgment is often excellent. It is a recommendation to make the trust model as rigorous as the numerical and formal machinery.

The most valuable improvement is a **machine-readable assurance graph**. Every claim should be a node; every translation or validation step should be a typed edge with an exact artifact hash, evidence class, reviewer, status, and limitation. README tables, badges, release notes, PDF cover capsules, and review ledgers should be generated from that graph. A claim is not "green" because a long narrative exists; it is green only when all required edges are closed. This single redesign would remove much of the present scope ambiguity and documentation sprawl.

1.1 Highest-priority conclusions

1. **Release-blocking truth-in-labeling defect.** The release and README say "reviewed source"; the ledger explicitly says no line review was inferred and no independent or human review was claimed for all 186 rows.
2. **Assurance evidence is real but non-transitive.** Lean proofs, MPFR enclosures, Z3 obligations, Decimal oracles, and Rust regressions verify different hand-selected objects. They do not automatically prove paper correspondence, bytes-to-counts, formal-to-Rust refinement, binary64 global error, estimator consistency, or application validity.
3. **The repo's source criticism is often correct.** The Ehrlich paper defects identified in Equation (8), the post-Definition-2 differential, Equation (14), and Algorithm 6 are genuine. The Schick-Poland normalization, bicontinuity, null-RCP, and Borel-isomorphism objections are also serious, but the Cantor-law wording should be narrowed.
4. **The categorical SxPID core is the strongest scientific surface.** Its signed nature, pointwise/averaged distinction, XOR behavior, and separation from Williams-Beer `I_min` are handled carefully.
5. **The continuous composition needs stronger uncertainty semantics.** Algebraically reconstructing four atoms from one redundancy estimate and three separately estimated mutual informations does not validate the atoms; a joint covariance/bootstrap object should be the default output.
6. **The deprecated Python compatibility path is unsafe epistemically.** It flattens structured results and loses quantization/sample-role provenance that the Rust API deliberately preserves.

7. **The documentation is too large to function as a review instrument.** the mathematical workflow document is 2,204 lines with 79 distinct external links; `METHODS.md` is about 488 kB with 73 method-detail blocks and extremely long evidence lines. More text has stopped meaning more auditability.

2 Scope, evidence, and limitations

2.1 Repository identity audited

- User-supplied URL: `sepehrmn/pid-rs`; GitHub currently redirects it to the canonical `sepahead/pid-rs`.
- Current head examined through the GitHub tree and file surfaces: `7e536f6` (exact commit).
- Published v0.9.0 tag/release tree identified by the repository ledger: `a9a2751` (exact commit).
- The head tree reported 162 commits at the time of review and contains a substantially expanded proof, claim, audit, script, PDF, Rust, Python, fuzzing, and release surface.

2.2 What I directly inspected

I directly inspected the visible current repository tree, every top-level entry, the complete core source and test inventories exposed by GitHub, the formal and output-PDF inventories, the full local copies of `MATHEMATICAL_PROBLEM_SOLVING_WORKFLOW.md`, `METHODS.md`, and `FILE_REVIEW_LEDGER.csv`, the key Rust and Python implementation bodies, the principal CI/dependency/release files, formal-audit documents, generated-paper sources and rendering receipts that were retrievable, and all load-bearing PID/MI primary sources listed in the workflow.

The mathematical review included equation- and algorithm-level checks of:

- Makkeh, Gutknecht, and Wibral categorical shared exclusions;
- Ehrlich et al. continuous shared exclusions and estimator pseudocode;
- Schick-Poland et al. measure-theoretic construction;
- Williams and Beer `I_min`;
- Kraskov-Stoegbauer-Grassberger mutual information;
- the newer Shannon-invariants, mixed-variable, multivariate-inconsistency, and structural-impossibility papers.

I also inspected the repository's own theorem and proof-boundary prose rather than treating formal tooling or test success as self-authenticating.

2.3 Important limitation: no false claim of complete byte review

A direct `git clone` of the exact head failed in this environment because outbound DNS resolution was unavailable. GitHub's web surface allowed extensive file-level review but did not expose every current blob byte in one retrievable archive. The repository's PDF blobs were served to the available transport as unsupported `application/octet-stream`, so I could not independently render and visually inspect those committed PDF binaries. I reviewed their Markdown/LaTeX source, output directory inventory, and the available rendering receipt instead.

Accordingly, this report is a deep, direct, risk-weighted audit, **not** a false statement that every byte of current `main` or every committed PDF page was independently inspected. That limitation is itself relevant: an assurance-focused release should make exact source archives and renderable artifacts straightforward for reviewers to retrieve.

2.4 Evidence grades used here

Grade	Meaning
A	Direct contradiction, exact source/formula check, or machine-counted repository record.
B	Strong code/design finding supported by inspected implementation and public contract.
C	Assurance, maintainability, or scientific-risk recommendation; not necessarily a current wrong result.

Grade	Meaning
U	Unverified because the required external code or binary artifact was inaccessible.

Severity is separate from confidence. “Release blocker” means a claim or contract should be fixed before 1.0; it does not mean a remotely exploitable security vulnerability.

3 Prioritized findings

RB-01 - Release blocker; confidence A. “Reviewed source” contradicts the 186-row all-unreviewed ledger.

RB-02 - Release blocker; confidence A/B. v0.9.0 tag, current `main`, generated papers, and assurance claims are not consistently bound in one visible state record.

RB-03 - Release blocker; confidence A/B. “Certified” and “formal assurance” artifacts do not close the paper-to-Rust correspondence chain; caveats exist but are too easy to miss.

H-01 - High; confidence A. Source errata are scientifically important but buried in a 2,204-line process document rather than encoded as first-class data and regression obligations.

H-02 - High; confidence A/B. Deprecated Python flat adapters discard quantizer/sample provenance and can reopen leakage or estimand-confusion risks.

H-03 - High; confidence A/C. Unrelated AI, Erdős, zeta, and social-media case studies contaminate the PID scientific trust surface and make the protocol harder to audit.

H-04 - High; confidence B/U. Generated PDFs lack a uniform, front-page assurance capsule and adjacent reproducibility/visual-review receipt; binary pages could not be independently rendered here.

H-05 - High; confidence A/C. Evidence is mostly implementation-separated, not institutionally independent; 1.0 needs named external scientific and code review.

H-06 - High; confidence U/B. The Ehrlich authors’ pinned reference code is load-bearing for intended wiring/units but was not retrievable; pid-rs needs a vendored minimal fixture or immutable source snapshot with license provenance.

M-01 - Medium; confidence A. `METHODS.md` is generated but nearly unauditable as prose: about 488 kB, 73 details, and giant single-line evidence fields.

M-02 - Medium; confidence B. Terms such as “semantic authority” and “review ledger” overstate hashes/inventories unless scope is explicit at the use site.

M-03 - Medium; confidence B. Nats/bits and evidence-state distinctions are represented mainly by naming/prose rather than strongly typed public values.

M-04 - Medium; confidence B/C. Continuous PID2 needs joint uncertainty and covariance by default; atom reconstruction alone is not statistical validation.

M-05 - Medium; confidence B/C. Custom PCA/IRLS/PLS numerics need richer residual, conditioning, and mature-solver oracle coverage.

M-06 - Medium; confidence A/C. Dependabot suppresses routine update PRs with a zero routine pull-request limit; security updates remain, but compatibility drift is deferred.

M-07 - Medium; confidence A/C. The main CI workflow is roughly 1,195 lines, increasing audit and mutation-risk despite strong individual controls.

M-08 - Medium; confidence B. “Independent” should always name the independence vector: semantic, implementation, custody, institutional, or data.

M-09 - Medium; confidence A. The Schick-Poland/Cantor critique is directionally right but overbroad; distinguish failure of the cited argument from impossibility of every mod-null representation.

M-10 - Medium; confidence A/C. A 2026 structural-impossibility preprint must remain visibly a preprint, not settled field consensus.

M-11 - Medium; confidence B. Two equal-width quantizer paths have different boundary semantics; generic naming plus legacy flattening is hazardous.

M-12 - Medium; confidence C. Narrative volume has replaced a compact dependency graph; review completion cannot be inferred reliably from prose density.

4 Detailed findings and required fixes

4.1 RB-01 - The release's "reviewed source" claim is contradicted by its own ledger

4.1.1 Evidence

The current README states that v0.9.0 "provides the exact reviewed source." The GitHub release similarly describes a reviewed source archive. Yet every one of the 186 rows in `FILE_REVIEW_LEDGER.csv` has:

- reviewer: `UNASSIGNED`;
- review status: `INVENTORIED_NOT_REVIEWED`;
- assumption: "exact-tag tree inventory only; no line review inferred";
- disposition: "inventory only; independent or human review not claimed."

This is not a subjective disagreement. It is an exact semantic contradiction between the public headline and the evidence record.

4.1.2 Why it matters

A reviewer encountering "reviewed source" reasonably infers that someone reviewed the source. A complete immutable inventory is valuable, but it is not a code review. The current phrasing can cause downstream users to upgrade an inventory into assurance that the repository explicitly disclaims in the CSV.

4.1.3 Required patch

Replace the README/release text with something like:

> Version 0.9.0 is a GitHub-only **source-review candidate**. It provides the exact source offered for review, an immutable tag inventory, proposed 1.0 scope records, and checksums. The included inventory does not claim that any file has completed independent or human line review.

Rename the artifact:

```
FILE_REVIEW_LEDGER.csv  ->  TAG_FILE_INVENTORY.csv
```

Create a true review ledger only when it has fields such as:

```
scope_commit,path,blob_sha,reviewer_identity,review_type,status,
findings,evidence,completed_at,review_signature
```

4.1.4 Acceptance test

CI must reject the words `reviewed`, `audited`, or `certified` in release-facing text unless a machine check proves that the referenced scope and required reviewer classes have completed dispositions. The check must fail on the current all-unreviewed CSV.

4.2 RB-02 - Scope binding is fragmented across tag, current main, claims, and generated artifacts

The repository now has at least three materially different scopes:

1. the v0.9.0 release/tag at `a9a275157237999c8da6ab813130d74f6113dec9`;
2. the much larger current head at `7e536f65822eecfb3f477cd675b857dc3fa3c373`;
3. generated assurance papers, proof projects, fixtures, and PDFs, each with its own source and toolchain state.

A reader can easily move from a tag-level release statement to current-main documentation, or from a current PDF to an older claim record, without seeing a single authoritative state capsule.

4.2.1 Required patch: `AUDIT_STATE.json`

Add one generated root file, validated on every merge and release:

```
{
  "repository": "sepahead/pid-rs",
  "scope_commit": "7e536f65822eecfb3f477cd675b857dc3fa3c373",
  "release_tag": null,
  "generated_at_utc": "2026-08-12T00:00:00Z",
  "review_inventory": {
    "artifact": "audit/TAG_FILE_INVENTORY.csv",
    "scope_commit": "a9a275157237999c8da6ab813130d74f6113dec9",
    "reviewed_files": 0,
    "inventoried_files": 186
  },
  "claim_registry_sha256": "...",
  "method_catalog_sha256": "...",
  "pdf_manifest_sha256": "...",
  "formal_projects": [
    {"name": "two-source-count-to-atom", "commit": "...", "status": "model-checked"}
  ]
}
```

Every generated PDF and release note should quote the relevant state record. A stale or cross-commit artifact should fail CI.

4.3 RB-03 - The correspondence ladder is open even where formal and numerical assurance is strong

The README and formal-audit prose correctly admit major exclusions: event and paper-facing semantics are a reviewed transcription; publication-to-Lean correspondence is not proved; bytes/rows-to-counts, Rust, binary64, parser/certifier execution, higher-source transfer, and population validity remain out of scope. Those caveats are intellectually honest. The problem is presentation: titles such as "certified executable assurance" or dense tables can dominate the caveat.

4.3.1 The five-edge ladder

Every scientific executable claim should be represented as these distinct edges:

1. **Source paper -> repository mathematical specification.** Were formulas, units, domains, and errata transcribed correctly?
2. **Repository specification -> formal model.** Does Lean/Z3 encode the intended repository statement?
3. **Formal model -> executable algorithm.** Does the formal object cover all branches, indexing, support logic, and termination conditions?
4. **Executable algorithm -> Rust/Python implementation.** Is there a refinement relation, or only bounded fixtures?
5. **Implementation output -> scientific estimand/application.** Are sampling, support, calibration, and domain assumptions justified?

The project has strong evidence on selected subsegments, especially exact count algebra and bounded cross-checks. It does not have one transitive proof over all five edges.

4.3.2 Required patch: assurance graph

Create `assurance/graph.jsonl`, with one record per node or edge:

```
{
  "id": "SX2-COUNT-ATOM-RUST-BINARY64",
  "kind": "edge",
  "from": "SX2-COUNT-ATOM-SPEC",
  "to": "pid_core::sxpid2",
  "scope_commit": "7e536f65822eecfb3f477cd675b857dc3fa3c373",
  "evidence_class": "bounded-conformance",
  "status": "partial",
```

```
  "artifacts": ["crates/pid-core/tests/..."],
  "reviewers": [],
  "excludes": [
    "global floating-point bound",
    "paper-to-spec correspondence",
    "population validity"
  ]
}
```

Generate all assurance language from this graph. Reserve `certified` for a named object and exact edge set, for example: "MPFR-enclosed evaluation of 24 coordinates from supplied positive-total count tables under specification revision X." Never let the word stand alone.

4.4 H-01 - Source errata need to become executable, first-class repository data

The workflow contains several important source corrections. They are among the most valuable parts of the repository, but they are embedded in prose near the beginning of a 2,204-line process compendium.

Create `audit/source-errata.json` with records like:

```
{
  "source": "arXiv:2311.06373v3",
  "location": {"physical_pdf_page": 28, "object": "Algorithm 6"},
  "observed": "target count calls _compute_n_alpha(T, antichain, eps)",
  "proposed_correction": "call _compute_n_T(T, eps)",
  "status": "reviewer-derived; not author-confirmed",
  "implementation_effect": "pid-rs already uses target-count routine",
  "tests": ["..."],
  "evidence_sha256": "..."
}
```

The implementation and tests should refer to erratum IDs. CI should fail if a load-bearing erratum loses its test or source pin.

4.5 H-02 - Python legacy flattening discards provenance that the Rust design treats as safety-critical

The structured Rust and newer Python surfaces preserve quantizer identity, sample role, interpretation, and fitted-transform provenance. The deprecated flat compatibility functions instead return bare dictionaries and preserve at most a bin count. That is not merely an ergonomic downgrade. It can erase whether boundaries were fit on the evaluation sample, which quantizer semantics were used, and whether the result is descriptive or intended as an estimator.

4.5.1 Required patch

For 1.0, remove the flat adapters. Before removal, make provenance loss an explicit opt-in error boundary:

```
def pid2_flat(*args, allow_provenance_loss: bool = False, **kwargs):
    if not allow_provenance_loss:
        raise ProvenanceLossError(
            "Flat compatibility output discards transform identity, sample role, "
            "and interpretation metadata. Use the structured result API."
        )
    return _legacy_flatten(pid2(*args, **kwargs))
```

Never return only `num_bins` for a fitted quantizer. Return immutable edges, algorithm revision, fit-row digest, input-shape digest, and role (`same_sample_descriptive`, `training_fitted`, or `externally_specified`).

4.5.2 Acceptance test

A serialized result must be sufficient to decide, without caller memory, whether evaluation rows contributed to transform fitting and which boundary rule produced every category.

4.6 H-03 - The workflow mixes PID assurance with unrelated process narratives

`MATHEMATICAL_PROBLEM_SOLVING_WORKFLOW.md` contains 79 distinct external links. Fourteen are the load-bearing PID/MI primary sources at its beginning. Much of the remainder concerns AI process threads, open Erdős problems, a cubic claim dispute, zeta-function work, model transcripts, social-media custody, and formal-tool case studies.

Those examples may be interesting research-process material, but they do not strengthen PID correctness. Their inclusion creates three problems:

- mutable or socially hosted material becomes part of the perceived trust surface;
- a reviewer must traverse hundreds of irrelevant pages before reaching the PID-specific protocol;
- process provenance can be mistaken for scientific evidence.

4.6.1 Required split

1. `PID_MATHEMATICAL_AUDIT_PROTOCOL.md` - at most about 30 pages, only PID definitions, source errata, claim graph, proof obligations, estimator assumptions, and acceptance rules.
2. `RESEARCH_PROCESS_CASE_STUDIES.md` - optional AI/Erdős/zeta examples, explicitly non-authoritative for PID.
3. `source-errata.json` and `source-registry.json` - machine-readable source roles, pins, hashes, and review status.

The compact PID protocol should be the only document linked from the stable API and release checklist.

4.7 H-04 - PDF publication contracts are not uniform or independently inspectable

The current output directory lists ten generated PDFs, including papers whose filenames contain `certified`, `assurance`, `audit`, `convergence`, `continuity`, and `workflow`. I could inspect the source documents and one rendering receipt, but the committed PDF blobs were not renderable through the available transport. Only the workflow PDF had an adjacent, visible rendering receipt in the directory inventory.

The available workflow receipt reports 64 pages, 120 DPI raster hashes, and luma/nonblank checks. That is useful deterministic transport evidence. It does **not** establish semantic correctness, absence of subtle clipping, correct equation layout, or human visual inspection.

4.7.1 Required PDF contract

Every PDF should have:

- a first-page assurance capsule;
- an adjacent `*.manifest.json` containing source hashes, exact commit, generator/toolchain versions, page count, render hashes, and build command;
- a `*.visual-review.json` naming the reviewer, renderer(s), date, pages inspected, and findings;
- a source-to-PDF text comparison for canonical reproduced sections;
- a release attachment rather than reliance on a browser blob path alone.

Suggested first-page capsule:

```
Evidence class: project analysis / exact-real proof / bounded executable check
Scope commit: <40-hex SHA>
Source files and SHA-256: <manifest link>
Peer review: none / internal / external named reviewers
What this establishes: <one paragraph>
What it does not establish: paper correspondence, Rust refinement, binary64 global error,
population validity, application validity (select exact list)
```

Rename ambiguous files, for example:

```
certified-sxpid2-executable-assurance.pdf
-> conditional-mpfr-assurance-for-supplied-sxpid2-count-tables.pdf
```

4.8 H-05 - Implementation separation is not the same as independent review

The repository often uses multiple routes: Lean, Z3, exact rational Python, high-precision Decimal, MPFR, Rust fixtures, and mutation tests. This is valuable fault diversity. But these routes often share:

- one maintainer;
- one semantic transcription;
- one selected theorem statement;
- one claim registry and repository custody chain;
- correlated fixtures or source-derived constants.

Call this **implementation-separated evidence**, not simply independent evidence. Before 1.0, obtain at least:

- one external PID-domain review of source transcription and method boundaries;
- one external Rust/numerics review of public stable paths;
- one independent rebuild/replay from a clean environment;
- signed reviewer dispositions bound to exact blobs.

The repository should record an independence vector:

```
semantic: shared / independent
implementation: shared / independent
institutional: shared / independent
custody: shared / independent
data: shared / independent
```

4.9 H-06 - The continuous-paper reference code is not a sufficient durable source pin

The workflow says the authors' pinned code disambiguates Algorithm 6 wiring and the bits conversion. That is plausible and consistent with the paper corrections. During this audit, the GitLab source was not retrievable without its unavailable transport/login path, so I could not independently confirm the exact code lines.

Do not make a stable implementation depend on a fragile external repository link. Subject to licensing, retain one of:

- a vendored minimal upstream fixture and exact source excerpt;
- a cryptographically hashed source archive in a release asset;
- a Software Heritage identifier;
- a small clean-room executable pseudocode specification with author-confirmed errata status.

Keep "authors' code indicates" separate from "paper states" and from "pid-rs implements."

4.10 M-01 and M-02 - Generated method catalog is complete but not human-reviewable

`METHODS.md` is generated from `method-catalog.json`, which is good for consistency. The resulting document is about 488 kB and 1,775 physical lines, with 73 method-detail sections. Many bounded-validation fields occupy one enormous physical line containing dozens of evidence paths and caveats.

A hash-checked generated artifact is a **catalog integrity snapshot**, not semantic authority. The generator can consistently render a wrong or stale source record.

4.10.1 Required presentation

Generate two outputs:

- `METHODS_SUMMARY.md`: one compact table, stable methods first, no cell over about 120 words;
- `METHOD_EVIDENCE.md` or browsable HTML: one evidence record per paragraph/path with anchors and filters.

Add schema-level fields instead of prose concatenation:

```
{
  "claim_status": "bounded",
  "evidence_classes": ["paper", "property-test", "formal-model"],
  "exclusions": ["population-validity", "rust-refinement"],
  "review_status": "internal-unassigned",
  "source_errata_ids": ["EHRLICH-A6-COUNT-ROUTINE"]
}
```

4.11 M-03 - Unit safety should be structural, not lexical

The core reports nats and often uses `_nats` suffixes, while papers and fixtures sometimes use bits. The Ehrlich Equation (14) defect shows why unit errors are easy and consequential. Introduce public newtypes:

```
#[repr(transparent)]
#[derive(Clone, Copy, Debug, PartialEq, PartialOrd)]
pub struct Nats(f64);

#[repr(transparent)]
#[derive(Clone, Copy, Debug, PartialEq, PartialOrd)]
pub struct Bits(f64);

impl Nats {
    pub fn to_bits(self) -> Bits { Bits(self.0 / std::f64::consts::LN_2) }
}

impl Bits {
    pub fn to_nats(self) -> Nats { Nats(self.0 * std::f64::consts::LN_2) }
}
```

Do not implement cross-unit `Add` or `Sub`. Serialize `{value, unit}` rather than relying only on a field name.

4.12 M-04 - Continuous PID2 requires joint uncertainty

The two-source identity

$$I(S_1, S_2; T) = R + U_1 + U_2 + S$$

is algebra. In the continuous implementation, `R` comes from the Ehrlich shared-exclusions estimator and the mutual-information coordinates come from separate KSG calls. These estimators have correlated sampling error, distinct bias, and potentially different support failures. Reconstructing atoms exactly from four noisy coordinates neither proves nonnegativity nor validates the scientific PID.

The default report should contain the full estimate vector and joint resampling object:

```
[R, I(S1;T), I(S2;T), I(S1,S2;T)]
bootstrap/permutation replicate matrix
4x4 covariance or robust uncertainty summary
atom replicate matrix after algebraic transformation
failure counts by coordinate and replicate
```

Do not create atom intervals by combining marginal endpoints. Transform each joint replicate. Flag strong cancellation, unresolved sign, and condition amplification.

4.13 M-05 - Numerical methods need diagnostic and oracle depth proportional to assurance claims

The custom bounded Jacobi PCA, Newton-IRLS logistic path, and PLS implementation appropriately fail on some nonconvergence conditions. To make them stable scientific infrastructure, add:

- relative residual and orthogonality diagnostics;
- condition/rank estimates and scale warnings;
- adversarial nearly singular, repeated-eigenvalue, separated-scale, and overflow-adjacent fixtures;
- comparison against a mature LAPACK/nalgebra/scipy reference on bounded fixtures;
- deterministic thread/backend parity where promised;
- no claim that finite oracle agreement is a global numerical proof.

For PCA, report at least `||AV - VΛ||`, `||V^T V - I||`, reconstruction error, and number of sweeps. A hard-coded sweep cap alone is not an accuracy contract.

4.14 M-06 and M-07 - Dependency and CI governance

Dependabot is configured weekly but sets a zero routine pull-request limit for Cargo, Actions, and pip. This still permits security updates, but routine compatibility drift accumulates. Keep the quiet main branch if

desired, but add a scheduled dependency-refresh branch/job that opens one aggregate compatibility report and runs the full matrix.

The principal CI workflow is roughly 1,195 lines. Its pins, permissions, negative tests, and no-credential choices are good. The size itself is a review risk. Move logic into versioned scripts or composite actions, then keep the workflow as a short orchestration graph. Add mutation tests for the release gates themselves and generate:

```
public claim -> required job IDs -> required artifacts -> exact scope commit
```

4.15 M-08 - Define "independent" every time

Use terms such as:

- `implementation-separated` for Python versus Rust code sharing semantics;
- `formalization-separated` for Lean versus handwritten derivation;
- `custody-separated` for artifacts signed and hosted independently;
- `institutionally independent` for unrelated reviewers;
- `data-independent` for genuinely held-out data.

A route can be independent along one dimension and correlated along another. The current prose often recognizes this eventually; put the vector at the first claim site.

4.16 M-09 - Narrow the Cantor-law objection

The repository is right that a bare Borel isomorphism does not establish the displayed atom-plus-Lebesgue-density decomposition in the original coordinate/reference measure. A singular-continuous Cantor distribution is an effective diagnostic against silently assuming ambient Lebesgue absolute continuity.

However, the stronger wording can be read as saying that no mod-null measure-space representation by Lebesgue measure exists. That is too broad: standard atomless probability spaces admit stronger measure-space isomorphism results modulo null sets. The correct criticism is:

> The cited Borel-isomorphism theorem and the original ambient coordinate/reference measure do not imply the displayed decomposition or the required density. A stronger mod-null measure-space isomorphism theorem would be a different argument and would not automatically preserve the local/topological constructions used later.

Change "Cantor-law counterexample" to "counterexample to ambient-coordinate absolute continuity and to inference from a bare Borel isomorphism."

4.17 M-10 - Keep recent impossibility work in the right evidentiary category

The 2026 Physical Review E paper on multivariate PID inconsistency is published and relevant to cross-subsystem consistency claims. The separate 2026 structural-impossibility manuscript is a current preprint. The repository is correct not to treat either as a direct code defect or an automatic refutation of categorical SxPID. Keep the preprint visibly labeled `preprint / external theoretical challenge`, record its exact revision, and avoid wording that implies settled consensus.

4.18 M-11 - Quantizer semantics need explicit versioned names

The repository documents two equal-width paths with different treatment of exact floating-point boundaries. One path reasons from exact-significand/sample semantics; another stores a stable edge vector. A value such as binary64 `0.3` can sit on different sides depending on how edges are generated and rounded.

Use explicit variants such as:

```
enum QuantizerSemantics {
    ExactSignificandEqualWidthV1,
    StoredBinary64EdgesV1,
}
```

Serialize the variant, exact edge bytes, fit-row identity, and revision. Do not offer a generic unqualified `equal_width` alias. This is especially important because the legacy Python path currently discards most of the distinction.

4.19 M-12 - Replace narrative assurance with a queryable claim graph

The repository has many excellent caveats, but they are distributed across README tables, `METHODS.md`, claim packets, formal audits, release documents, workflow prose, and PDFs. A user should be able to ask:

```
What exactly supports the claim that categorical SxPID2 output is correct?
Which edges remain open for continuous PID2 population use?
Which artifact was externally reviewed, by whom, at which commit?
```

The answer should come from one graph, not a literature search inside the repository. Generate human documents from the graph and add a linter that forbids a stronger verb than the weakest required edge permits.

5 Mathematical source audit

5.1 Ehrlich et al. continuous shared exclusions

Primary source: arXiv:2311.06373v3 and Physical Review E 110, 014115.

5.1.1 Equation (8): repeated source-bin factor

The paper's overlap display repeats `m_{S_2}` in the denominator where independent substitution of the two source-bin widths requires `m_{S_1}m_{S_2}`. The repository's correction is convincing for three independent reasons:

1. symmetry under exchanging the two sources;
2. dimensional bookkeeping of one width contribution from each source coordinate;
3. the surrounding derivation's use of separate source partitions.

This is best treated as a local source typo, not an estimator redesign.

5.1.2 Post-Definition-2 integral: repeated differential

The displayed global expectation uses `dt ds_1 ds_1`, although the integrand is a function of `(t,s_1,s_2)` and Appendix D uses `dt ds_1 ds_2`. The repository is correct: the second differential must be `ds_2`.

5.1.3 Equation (14): unit mismatch

The kNN expression is written with digamma functions and natural logarithms. Such an expression is in nats. The paper's definitions/examples are in bits, so the full right-hand side requires division by `ln 2` to report bits. `pid-rs`' choice to keep the computational core in nats and convert a bit-valued fixture by multiplying by `ln 2` is coherent. This should be enforced by types and an erratum-linked test.

5.1.4 Algorithm 6: omitted antichain and wrong target count routine

The printed pseudocode omits the antichain argument in source-disjunction calls and invokes the source-count routine on the target. The intended logic requires:

```
_compute_epsilons(S, T, antichain)
_compute_n_alpha(S, antichain, eps)
_compute_n_T(T, eps)
```

The repository's judgment is correct. Also correct are the smaller observations that Algorithm 5 advertises an unused `alpha` and Algorithm 6's result header describes distances despite returning a redundancy scalar.

5.1.5 What remains unproved

Correcting these defects does not establish consistency of the source-disjunction estimator, its gauge behavior, finite-sample calibration, support validity, mixed-variable generality, or equivalence to the Schick-Poland construction. `pid-rs` generally says this; the same limitation must be impossible to miss in the experimental API.

5.2 Schick-Poland et al. measure-theoretic shared exclusions

Primary source: arXiv:2106.12393v2.

5.2.1 Missing conditional normalization

For an event `R` with positive probability,

$$P(T \in A \mid R) = \frac{P(T \in A, R)}{P(R)}.$$

The finite-discrete recovery display appears to use the numerator without the denominator. Unless the authors intentionally define an unnormalized restricted measure and normalize later, exact recovery of the categorical construction has an open obligation. The repository appropriately calls this a source obligation rather than claiming an author-confirmed erratum.

5.2.2 Bimeasurable is not bicontinuous

A measurable bijection with measurable inverse is not automatically a homeomorphism. Any proof step requiring preservation of neighborhoods, local balls, limits, or topological density behavior needs continuity or a different invariant formulation. The repository's objection is correct.

5.2.3 RCP values at null conditioning events

Regular conditional probabilities are generally unique only almost everywhere with respect to the conditioning law. Evaluating a chosen version at a null indicator value is not version-invariant without a canonical version or additional regularity. This is a real well-definedness issue for pointwise local values.

5.2.4 Borel isomorphism and the Cantor nuance

A Borel isomorphism is a measurable-structure statement, not a proof of an atom-plus-Lebesgue density decomposition in the original coordinates. The repository is right on that central point. It should, however, avoid implying that stronger standard-probability-space isomorphism results modulo null sets are impossible. Such results do not rescue the displayed argument automatically, because they change the reference representation and may not preserve the local/topological structures the construction uses.

5.2.5 Implementation judgment

`pid-rs` correctly declines to implement a practical general mixed-variable estimator from this theory. That is a sound negative capability, not a missing feature to paper over.

5.3 Makkeh-Gutknecht-Wibral categorical SxPID

Primary source: arXiv:2002.03356v5 and Physical Review E 103, 032149.

The repository's most important conceptual judgment is to keep categorical shared exclusions separate from `I_min` and from continuous shared exclusions. The common PID vocabulary and antichain lattice do not provide a mapping theorem between redundancy functionals.

For XOR, shared-exclusions SxPID is signed. Its averaged redundancy is approximately `-0.58496` bits and its synergy approximately `1.58496` bits, rather than the simple `0` redundancy / `1` synergy associated with the familiar Williams-Beer account. The repository correctly records that the earlier simplistic XOR expectation was wrong. This is not a numerical nuisance; it expresses the method's informative/misinformative event semantics.

The foundational audit's conclusion is defensible: SxPID is a coherent signed, local, event-logical decomposition under its declared semantics. It is not thereby a unique ontology of redundancy, a causal attribution, a nonnegative set-size decomposition, or an application-authority measure.

Recommended addition: publish a small exact rational appendix for canonical AND, COPY, XOR, and noisy-XOR laws, including all cumulative informative/misinformative products before logarithms. This makes semantic disagreements visible before floating point enters.

5.4 Williams-Beer `I_min`

Primary source: arXiv:1004.2515v1.

The source defines the minimum-specific-information redundancy over the redundancy lattice and yields nonnegative partial-information atoms in its intended setting. It gives the familiar XOR synergy result.

Later identity/copy criticisms are conceptual limitations of the redundancy functional, not implementation bugs.

`pid-rs` is right to keep `I_min` as a named legacy comparator, not a default synonym for PID or SxPID. Every serialized result should continue to carry the method identity.

5.5 Kraskov-Stoegbauer-Grassberger mutual information

Primary source: arXiv:cond-mat/0305641v1, Physical Review E 69, 066138, and the 2011 Appendix A5 erratum.

The inspected `pid-rs` implementation follows the core KSG1 geometry in the important places:

- max norm in the joint space;
- strict marginal counts inside the kth joint radius;
- natural-log/digamma units;
- a next-down radius operation to implement strict binary64 membership.

Rejecting exact ties, shell ambiguity, and a nonzero user tie tolerance is stricter than the original paper. That is a defensible fail-closed project support policy. It should be labeled as such rather than attributed to the KSG theorem.

The 2011 erratum concerns an appendix extremum statement and says the other results are unaffected. The repository is right not to use that corrected appendix statement as an estimator-validity bridge.

The largest remaining risk is not the local count formula; it is overinterpreting a finite-sample estimate when continuity, support, finite MI, boundary, local-density, metric, or sampling assumptions have not been justified. The current support contract is appropriately conservative but should distinguish declaration from external proof.

5.6 Newer source routing

5.6.1 Shannon invariants

Gutknecht et al., arXiv:2504.15779v1 defines target-conditioned screening quantities. The repository correctly labels target-free entropy-ratio constructions as project-defined analogues rather than the published quantities. Encode this in distinct types/schema IDs.

5.6.2 Barà mixed-variable estimator

Barà et al., arXiv:2409.13506v1 treats a narrower mixed setting, notably a discrete target with continuous sources and KL/kNN machinery. It is not a general estimator for the Schick-Poland shared-exclusions functional. The repository's nonimplementation statement is correct.

5.6.3 Multivariate inconsistencies and structural impossibility

Lyu, Clark, and Raviv, arXiv:2508.05530v2, published in Physical Review E in 2026, identifies incompatibilities among desired multivariate PID properties. It does not by itself prove a software defect in categorical SxPID. The structural-impossibility preprint, arXiv:2604.03869v2, is relevant but should remain labeled a preprint and exact revision.

6 Formal methods and proof-boundary audit

6.1 What is genuinely strong

The formal documents repeatedly state what they do not prove. The count-to-atom bridge fixes exact positive-total counts, reconstructs all 24 two-source informative, misinformative, and signed-net cumulative/atom coordinates, proves finite Möbius inversion identities, and reduces sign/zero questions to exact product comparisons. Other Lean projects check deterministic continuity cores, finite keyed-event maps, and algebraic bounds. Z3 and mutation tests add useful negative adequacy checks. High-precision generators and Rust replays create implementation-separated conformance evidence.

This is serious work. The project is correct not to present a solver answer, same-repository digest, or fixture pass as authenticity or universal mathematical truth.

6.2 What remains open

The dominant open obligations are:

- paper text/equations -> repository semantic transcription;
- rows/bytes -> categorical events/counts;
- full branch/index coverage of the Rust implementation;
- exact-real theorem -> binary64 result with a global error bound;
- high-precision reference -> independent implementation rather than correlated formulas;
- finite fixtures -> universal refinement;
- deterministic functional -> sampling consistency/calibration;
- estimator output -> domain/application validity.

6.3 Naming rules

Use the strongest accurate noun phrase, not the strongest impressive adjective. Examples:

Current style	Better style
certified SxPID2 executable assurance	MPFR enclosures for 24 SxPID2 coordinates from supplied count tables
independent oracle	implementation-separated exact-rational fixture generator
semantic authority	hash-checked catalog snapshot
reviewed source	source offered for review / immutable inventory
formal proof of the implementation	formal proof of named model theorem; Rust refinement open

6.4 Proof adequacy tests

Every formal project should include three separate mutation classes:

1. **Statement mutations:** plausible but wrong theorem statements that must not be accepted as satisfying the claim.
2. **Correspondence mutations:** swapped indices, wrong event complement, wrong logarithm base, omitted normalization, wrong antichain argument.
3. **Implementation mutations:** branch inversions, shell comparator changes, count routine swaps, serialization/provenance erasure.

The repository already does parts of this. The claim graph should show which mutation classes each artifact kills.

7 PDF and Markdown workflow audit

7.1 Mathematical workflow document

Measured local source:

- 2,204 lines;
- 167,632 bytes;
- 79 distinct external links;
- PID-specific source pins and errata concentrated near the opening and later PID application sections;
- a large middle body of unrelated mathematical and AI-process case studies.

The document's central rule - retained and replayable evidence, not model confidence - is sound. Its distinctions between proposal, falsifier, proof, oracle, custody, and external review are also strong. The problem is editorial architecture. A mandatory PID protocol should not require readers to audit mutable social posts, large model transcripts, open-problem narratives, or unrelated formal projects.

The associated LaTeX/PDF companion says it reproduces the canonical text byte-for-byte and adds a primer/evidence supplement. That helps source traceability, but it also turns an already huge document into a 64-page artifact. Split the canonical source first; typesetting cannot solve conceptual overloading.

7.2 METHODS.md

Measured local source:

- 1,775 lines;
- 487,683 bytes;
- 45 distinct external source links;
- 73 generated method-detail sections;
- strong method/paper/project distinctions;
- many enormous bounded-validation paragraphs and evidence-path lists.

The machine-readable catalog should remain canonical for structured facts. The Markdown should become a generated view optimized for human decisions, not a lossless dump of every field.

Recommended views:

- stable public methods;
- experimental/research methods;
- unsupported methods;
- source errata dependencies;
- assurance status by five-edge ladder;
- application assumptions;
- exact change since last release.

7.3 Generated PDF set

The current output directory lists:

1. `certified-sxpid2-executable-assurance.pdf`
2. `dependency-colored-sxpid-concentration.pdf`
3. `ecosystem-compatibility-audit.pdf`
4. `exact-log-product-sxpid2-assurance.pdf`
5. `finite-alphabet-plugin-convergence.pdf`
6. `formal-tool-adoption-audit.pdf`
7. `foundational-shared-exclusions-pid-audit.pdf`
8. `mathematical-problem-solving-workflow.pdf`
9. `support-change-tolerant-averaged-sxpid-continuity.pdf`
10. `two-source-sxpid-count-atom-bridge.pdf`

For each, add a one-row README index with exact source, commit, evidence class, reviewer class, peer-review status, and out-of-scope list. Keep project theorem papers explicitly labeled "unpublished project analysis; not externally peer reviewed" unless that changes.

8 Module-by-module code review matrix

The records below cover every current core module visible in the repository inventory and the adjacent public surfaces. "Strong" means the inspected design choice is sound within its stated scope, not that every byte or scientific claim has been independently certified.

`bootstrap.rs`
Judgment: Strong resource and cancellation contracts.
Improvement: Separate descriptive resampling from estimator-specific calibration; require the estimand and failure-retention policy in every result.

`ci.rs`
Judgment: Useful algebraic reconstruction and confidence summaries.
Improvement: For continuous PID2, make joint resampling of redundancy and all MI coordinates the primary uncertainty path; expose covariance, not just marginal intervals.

`discrete_pid.rs`
Judgment: Good separation of categorical SxPID and legacy I_min.
Improvement: Use unit-safe result types and keep method identity in serialized output. Add exact rational regression vectors for every public lattice size.

`distance_matrix.rs`
Judgment: Checked size arithmetic and a clear diagnostic role.

Improvement: State O(n^2) memory at the public entry point and prevent accidental use as the default neighbour backend for large samples.

error.rs
Judgment: Typed errors are a strength.
Improvement: Add stable machine-readable error codes and causal chaining so Python and run logs do not collapse distinct support, resource, and numerical failures.

geometry.rs
Judgment: Diagnostics are appropriately separated from estimator validity.
Improvement: Make it impossible for a geometry diagnostic alone to promote KSG support from unverified to satisfied.

hierarchy.rs
Judgment: Exploratory hierarchy tooling is useful.
Improvement: Keep it visibly exploratory and prohibit inferential language unless a method-specific calibration is supplied.

hyperbolic.rs
Judgment: Metric adaptation is explicit.
Improvement: State that metric construction does not establish consistency of a PID or MI estimator; require backend parity tests at shell boundaries.

identity.rs
Judgment: Correctly states that hashes are not authentication.
Improvement: Generate software identity from one signed/attested build manifest and validate freshness at compile/package time rather than maintaining parallel prose.

invariants.rs
Judgment: Good distinction between published target-conditioned quantities and project analogues.
Improvement: Encode that distinction in the type/schema name, not only prose and field suffixes.

isx.rs
Judgment: The experimental status and source caveats are mostly honest.
Improvement: Drive the implementation from a machine-readable source-errata record; add an executable trace showing every Algorithm 6 count and unit conversion.

kdtree.rs
Judgment: Exact-neighbour intent and resource checks are strengths.
Improvement: Add differential shell/tie tests against a brute-force backend over adversarial nextafter inputs and repeated coordinates.

ksg.rs
Judgment: One of the strongest modules: natural-log units, max norm, strict marginal counts, tie rejection.
Improvement: Report the exact support-contract disposition and tie/shell diagnostics with every estimate. Keep fail-closed policy labeled as project policy, not a paper theorem.

lib.rs
Judgment: Feature gates and unsupported routes are explicit.
Improvement: Reduce the exported surface before 1.0 and make experimental modules impossible to confuse with stable methods in generated documentation.

logistic.rs
Judgment: Convergence failure is surfaced rather than silently accepted.
Improvement: Expose iteration count, gradient norm, Hessian conditioning proxy, regularization, and a reference-solver oracle suite.

matrix.rs
Judgment: Checked constructors and non-panicking alternatives are good.
Improvement: Deprecate or confine panicking indexing helpers on user-controlled dimensions; prefer Result-returning views at public boundaries.

metric.rs
Judgment: Metric semantics are centralized.
Improvement: Version and serialize metric/norm conventions because KSG shell membership is part of the estimator definition.

nn.rs
Judgment: Backend abstraction is useful.

Improvement: Require conformance fixtures proving identical kth radius and strict marginal counts across all exact backends.

observation.rs
Judgment: Provenance-bearing observations are the right direction.
Improvement: Make training/evaluation role and transform identity non-optional for every fitted transform path.

par.rs
Judgment: Parallelism is treated as an engineering layer.
Improvement: Document deterministic reduction guarantees and test bitwise or tolerance-bounded parity across thread counts.

pid2.rs
Judgment: Algebraic decomposition is clear and method identities are mostly retained.
Improvement: Do not let reconstruction identity read as validation. Return a joint uncertainty object and an explicit estimator-combination warning.

pid3.rs
Judgment: Incomplete versus full/research paths are distinguished.
Improvement: Make incomplete output a different top-level type with no conversion to a complete PID without an explicit failing proof obligation.

pipeline.rs
Judgment: The repository recognizes fit/evaluation leakage risks.
Improvement: Represent fit scope in types: TrainingFitted, SameSampleDescriptive, and ExternallySpecified. Do not infer role from call sequence.

pls.rs
Judgment: Strong leakage warning and explicit convergence behavior.
Improvement: Add independent numerical oracle tests and serialize training-row identity/fold assignment in the fitted object.

preprocess.rs
Judgment: Bounded custom Jacobi PCA fails rather than fabricating convergence.
Improvement: Add residual, orthogonality, condition, and reconstruction diagnostics plus adversarial comparison with a mature eigensolver.

quantizer.rs
Judgment: Quantization provenance is recognized.
Improvement: Rename and version the two boundary semantics; do not expose both under a generic equal-width label. Serialize edge-generation revision.

report.rs
Judgment: Rich claim-boundary reporting is a major strength.
Improvement: Split numeric estimate, evidence status, and scientific interpretation into separate nested objects so downstream code cannot discard warnings accidentally.

resource.rs
Judgment: Pervasive checked preflight is exemplary.
Improvement: State that preflight is not an allocator guarantee. Record requested/approved budgets and actual high-water estimates in reports.

same_sample.rs
Judgment: Correctly marks a descriptive same-sample route.
Improvement: Use an unmistakable type/name and prohibit conversion into a population-estimator report without an explicit unsafe-inference acknowledgement.

stats.rs
Judgment: Multiplicity and typed null mechanisms are valuable.
Improvement: Require the family definition, dependence assumption, retained failures, and correction target in serialized output.

support.rs
Judgment: Fail-closed support declarations are much better than silent assumptions.
Improvement: Distinguish declared, diagnostically compatible, externally justified, and theorem-satisfied states. A declaration is not proof.

`sxpid.rs`
Judgment: The categorical core, event semantics, signed atoms, and pointwise/averaged split are unusually careful.
Improvement: Add interval/high-precision cross-checks to public canonical fixtures and bind each formula to a source-errata/semantic claim ID.

`Python bindings`
Judgment: C-contiguity checks prevent silent Fortran-order reinterpretation.
Improvement: Delete or gate flat legacy adapters that discard quantizer and sample-role provenance; return structured result classes only.

`Run logs`
Judgment: The non-authentication disclaimer is correct.
Improvement: Support optional signatures/attestations and an external custody chain; never promote a same-repository digest to authenticity.

`CI/workflows`
Judgment: Pinned actions, narrow permissions, no persisted credentials, and many negative tests are strong.
Improvement: Break the 1,195-line workflow into checked scripts/composite actions; generate a claim-to-job map and test that every release claim points to a current passing job.

9 Cross-cutting API and scientific-contract improvements

9.1 Separate numeric values from evidence and interpretation

A robust result schema should have three nested layers:

```
{
  "estimate": {"value": 0.123, "unit": "nat", "method": "ksg1", "revision": 1},
  "evidence": {
    "support_status": "declared-not-externally-justified",
    "numerical_status": "finite-binary64",
    "calibration_status": "not-provided",
    "scope_commit": "7e536f65822eecfb3f477cd675b857dc3fa3c373"
  },
  "interpretation": {
    "estimand": "...",
    "aggregation": "empirical-pmf-plugin",
    "exclusions": ["causality", "responsibility", "population-unbiasedness"]
  }
}
```

Do not expose convenience methods that return the number while silently dropping the other two layers without an explicit lossy operation.

9.2 Treat negative capability as a first-class feature

The repository is strongest when it says “no implementation” or “research only.” Preserve this. Add a stable `UnsupportedRequest` taxonomy with source-backed reasons. This is better than exposing a plausible-looking but unjustified mixed-variable or full continuous PID.

9.3 Make same-sample descriptive routes impossible to confuse with estimators

Use distinct top-level types and function names. A same-sample quantized result can be useful as a descriptive empirical functional. It should never serialize with the same method identifier as a training-fitted population-estimation attempt.

9.4 Add cancellation/conditioning diagnostics for signed atoms

For each signed atom `net = informative - misinformative`, report:

- both components;
- absolute and relative cancellation;
- a scale-aware unresolved-sign flag;

- high-precision or interval fallback for canonical/small tables;
- no zero clamping.

The repository already avoids clamping; make the diagnostic universal.

10 Testing strategy: what to retain and what to add

10.1 Retain

- strict finite/shape checks;
- resource-overflow and cancellation tests;
- C-contiguous NumPy enforcement;
- known-failure and mutation fixtures;
- exact categorical canonical tables;
- KSG tie/shell tests;
- cross-language bounded oracles;
- fail-closed unsupported paths;
- release credential and action-pin controls.

10.2 Add

1. **Source-erratum regression tests.** Each corrected paper formula/pseudocode item gets an ID and a minimal test that the wrong variant fails.
2. **Correspondence tests.** Machine-readable maps from paper symbols to repository fields and formal definitions.
3. **Quantizer boundary corpus.** `nextafter` values around every edge, signed zero, subnormals, huge ranges, repeated min/max, and serialization round trips.
4. **Backend parity.** Brute force versus tree neighbour radii/counts across adversarial datasets.
5. **Numerical oracle suite.** PCA/PLS/logistic against mature solvers, with residual thresholds and condition strata.
6. **Joint continuous-PID resampling.** Preserve replicate covariance and failure patterns.
7. **Documentation truth tests.** Mutate ledger statuses or scope SHAs and prove release claims fail generation.
8. **PDF visual regression.** Render every page in two engines for release PDFs, check blank/clipped/overflow conditions, and require named visual review.
9. **Consumer-loss tests.** Attempt to flatten/serialize results and assert that loss of provenance requires explicit opt-in.
10. **Formal-statement adequacy.** Wrong but type-correct theorem variants must not satisfy the claim registry.

11 CI, release, and supply-chain review

11.1 What is correct

The inspected workflows use pinned action revisions, narrow permissions, `persist-credentials: false`, explicit release boundaries, and many self-tests. The repository avoids claiming that a hash authenticates an artifact. These are mature choices.

11.2 What should change before 1.0

- Publish a signed annotated tag or Sigstore/GitHub attestation policy and document key/custody assumptions.
- Have at least two named external reviewers sign blob-bound dispositions.
- Generate SBOM and provenance for actual release artifacts, not for the source-review prerelease only.
- Make release reproduction start from a clean archive and produce a one-command evidence bundle.
- Separate tag inventory, code review, scientific review, formal review, and visual/PDF review.
- Require all release claims to be generated from `AUDIT_STATE.json` and the assurance graph.
- Keep routine dependency compatibility visible despite zero routine Dependabot PRs.
- Make PDF/source artifacts downloadable as release assets with correct `application/pdf` content type.

12 What the authors got right

A critical audit should preserve good judgment rather than flattening everything into defects.

1. **Safe implementation core.** `#![forbid(unsafe_code)]`, checked arithmetic, explicit resource budgeting, finite-value validation, and cooperative cancellation are excellent defaults.
2. **Method separation.** Categorical SxPID, continuous Ehrlich SxPID, `I_min`, KSG MI, heuristic routes, and mixed-variable theories are not silently merged.
3. **Signed semantics.** Informative, misinformative, and net atoms are retained; tiny negative binary64 residuals are not clamped away.
4. **Pointwise versus averaged types.** The repository recognizes that a pointwise realization value and an empirical-PMF average are different objects.
5. **Strict Python layout checking.** Refusing non-C-contiguous arrays prevents silent transposition or stride misinterpretation.
6. **KSG local fidelity.** Max-norm geometry, strict marginal counts, natural-log units, and tie/shell conservatism are handled carefully.
7. **Negative capabilities.** The library explicitly declines unsupported general mixed PID and warns about incomplete/full continuous PID3 routes.
8. **Formal caveats.** The formal audit repeatedly states that source correspondence, Rust refinement, binary64, and population validity remain outside named proofs.
9. **Run-log honesty.** Hashes are described as identity/integrity aids, not authentication.
10. **Source criticism.** The Ehrlich and Schick-Poland source defects were identified with better care than is typical in scientific software.
11. **No novelty laundering.** "Paper-defined," "paper-derived," and "project-defined" distinctions are generally used well.
12. **Application restraint.** Atoms are not presented as causal responsibility, deception, or authorization evidence by default.

13 A pre-1.0 remediation sequence

13.1 0.9.1 - Truth and scope

- Replace "reviewed source" language.
- Rename the inventory artifact.
- Add `AUDIT_STATE.json`.
- Bind every generated document to exact source and commit.
- Add a release-claim linter.

13.2 0.9.2 - Scientific correspondence

- Create `source-errata.json`.
- Create the five-edge assurance graph.
- Move PID source review into a concise protocol.
- Obtain external paper-transcription review.
- Add source-erratum mutation tests.

13.3 0.9.3 - API and numerics

- Remove/gate provenance-dropping Python APIs.
- Introduce `Nats`/`Bits` and explicit quantizer-semantics types.
- Make joint continuous-PID uncertainty the primary report.
- Add numerical residual/condition/oracle suites.
- Add backend and quantizer-boundary parity corpora.

13.4 1.0 - Reviewable release

A 1.0 release should include:

- exact source archive;
- signed/attested tag and build provenance;
- SBOM;

- named external code and mathematical review dispositions;
- generated assurance graph and state file;
- concise human methods summary;
- full machine catalog;
- source errata registry;
- uniformly manifested and visually reviewed PDFs;
- no stable API capable of silently discarding sample-role or transform provenance.

14 Concrete acceptance checklist

A reviewer should be able to answer yes to all of the following without reading hundreds of pages:

- Does every use of `reviewed`, `verified`, `formal`, `certified`, or `independent` identify an exact object, scope, evidence class, and reviewer class?
- Does the release headline agree with the machine ledger counts?
- Can every PDF be rebuilt and rendered from exact source with a named visual review?
- Can every paper correction be located in a source-errata record and a regression test?
- Can a user tell nats from bits from the type/schema alone?
- Can a user tell same-sample descriptive quantization from training-fitted evaluation from the result alone?
- Does every continuous PID2 uncertainty report preserve joint replicate dependence?
- Does every formal claim show open paper-correspondence and Rust-refinement edges?
- Are unsupported mixed/general routes impossible to call accidentally?
- Are external reviewers' dispositions bound to exact blobs and independently custodied?

15 Appendix A - v0.9.0 inventory metrics

The supplied `FILE_REVIEW_LEDGER.csv` contains **186** file rows.

- Reviewer values: `UNASSIGNED` = 186.
- Review statuses: `INVENTORIED_NOT_REVIEWED` = 186.
- Public-surface rows: 56.
- Security-critical rows: 61.
- Science-critical rows: 75.
- Authority-critical rows: 19.

Language inventory:

Language/category	Files
Rust	77
text	22
Markdown	21
JSON	11
shell	11
TOML	10
license text	8
Python	8
YAML	7
configuration	4
JSON Lines	3
SVG	2
Python type stub	1
Just	1

The inventory is useful and should be preserved. The defect is naming and headline interpretation, not the existence of the inventory.

16 Appendix B - External source inventory from the mathematical workflow

This appendix lists every distinct external link parsed from the inspected workflow source. The disposition is deliberately role-based. Only the PID-primary entries were treated as load-bearing scientific sources for equation/algorithm review. Process and unrelated mathematical case studies were reviewed for role and custody claims, not re-proved as part of this PID audit.

1. "Introducing a differentiable measure of pointwise shared information," arXiv:2002.03356v5 - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

2. Physical Review E 103, 032149 (2021) - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

3. "Partial Information Decomposition for Continuous Variables based on Shared Exclusions: Analytica... - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

4. Physical Review E 110, 014115 (2024) - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

5. "A partial information decomposition for discrete and continuous variables," arXiv:2106.12393v2 - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

6. "Nonnegative Decomposition of Multivariate Information," arXiv:1004.2515v1 - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

7. "Estimating Mutual Information," arXiv:cond-mat/0305641v1 - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

8. Physical Review E 69, 066138 (2004) - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

9. 2011 erratum to Appendix Eq. (A5) - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

10. "Shannon invariants: A scalable approach to information decomposition," arXiv:2504.15779v1 - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

11. "Partial information decomposition for mixed discrete and continuous random variables," arXiv:240... - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

12. "Multivariate Partial Information Decomposition: Constructions, Inconsistencies, and Alternative ... - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

13. Physical Review E 113, 034102 (2026) - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

14. "Structural Impossibility of Antichain-Lattice Partial Information Decomposition," arXiv:2604.038... - workflow line 151; **PID primary**. Equation/algorithm/claim-boundary review where load-bearing.

15. Thread introduction - workflow line 189; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

16. Exact problem and audit requirements - workflow line 190; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

17. Independent routes and blocker rules - workflow line 191; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

18. Long-run work pattern - workflow line 192; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

19. Research loop - workflow line 193; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

20. Cycle Double Cover prompt - workflow line 197; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

21. **f2ae0edb45cb...** - workflow line 215; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

22. **1002** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

23. **1038** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

24. **390** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

25. **486** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

26. **536** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

27. **788** - workflow line 221; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

28. **numerical verifier** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

29. **Lean project** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

30. **486** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

31. **788** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

32. **390** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

33. **536** - workflow line 238; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

34. **pw/erdos477-cubic** - workflow line 259; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

35. **8440d599890b...** - workflow line 259; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

36. **VERIFICATION.md** - workflow line 272; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

37. **cubic claim** - workflow line 276; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

38. **generalized claim** - workflow line 276; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

39. **original claim** - workflow line 301; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

40. **linked draft** - workflow line 301; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

41. **specific objection** - workflow line 301; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

42. **acknowledgement** - workflow line 301; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

43. **correction post** - workflow line 301; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

44. **2306.04631v3** - workflow line 333; **Supporting/process source**. Retain only with an explicit role; not evidence for PID correctness.

45. **1604.00365v2** - workflow line 360; **Supporting/process source**. Retain only with an explicit role; not evidence for PID correctness.

46. **Jacobian discussion** - workflow line 449; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

47. **unsplittable-flow discussion** - workflow line 456; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

48. **pid-rs correctness discussion** - workflow line 462; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

49. **How the Ideas Came Together (reasoning-walkthroughs.pdf)** - workflow line 471; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

50. **Ten Advances in Mathematics and Theoretical Computer Science** - workflow line 471; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

51. **Alpöge post** - workflow line 532; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

52. **note-and-image reply** - workflow line 532; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

53. **historical explanation** - workflow line 532; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

54. **Anthropic announcement** - workflow line 532; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

55. **X media rendition** - workflow line 532; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

56. **Anthropic research page** - workflow line 552; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

57. **More Than Two Thirds of the Zeros of the Riemann Zeta Function Lie on the Critical Line** - workflow line 552; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

58. **5-page informal note** - workflow line 552; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

59. **How the two-thirds argument was found: two agent runs and their literature** - workflow line 552; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

60. **116-page annotated transcripts** - workflow line 552; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

61. **Lean repository, annotated tag object 82ee6340d6fb... peeling to commit ...** - workflow line 552; **Formal-tool case study**. Process/custody example only; no PID evidentiary transfer.

62. **Mathlib 51e6992efd06...** - workflow line 552; **Formal-tool case study**. Process/custody example only; no PID evidentiary transfer.

63. **Comparator, reviewer-inspection commit 273294467ce0...** - workflow line 552; **Formal-tool case study**. Process/custody example only; no PID evidentiary transfer.

64. **arXiv:2306.04799v1** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

65. **exact PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

66. **arXiv:2501.14545v2** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

67. **exact PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

68. **arXiv:2511.20059v2** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

69. **exact PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

70. **arXiv:2603.28104v1** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

71. **exact PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

72. **arXiv:2503.15449v4** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

73. **exact PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

74. **"Remarks on Weil's quadratic functional in the theory of prime numbers"** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

75. **exact reviewed PDF transport** - workflow line 575; **Unrelated mathematical case study**. Role/custody reviewed only; move outside PID trust surface.

76. **non-primary contextual reply** - workflow line 584; **Mutable process narrative**. Do not use as PID scientific authority; archive or remove from core protocol.

77. **check-citation-edge-countermodel.py** - workflow line 1692; **Repository evidence**. Direct repository artifact; bind to an exact commit, not `main`.

78. **check-citation-edge-countermodel-self-test.py** - workflow line 1692; **Repository evidence**. Direct repository artifact; bind to an exact commit, not `main`.

79. **PidCitationEdgeCountermodel.lean** - workflow line 1702; **Repository evidence**. Direct repository artifact; bind to an exact commit, not `main`.

17 Appendix C - External source inventory from `METHODS.md`

`METHODS.md` repeats many links across method records; the table below lists all 45 distinct links. A catalog citation proves neither correct implementation nor applicability. Each method record should link to the exact assurance-graph edges that connect source, specification, implementation, and scientific assumptions.

1. **Anthony J. Bell (2003)** - catalog line 36; **Method-supporting source**. Role/provenance checked; no universal re-derivation claimed.

2. **William J. McGill (1954)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

3. **Alexander Kraskov et al. (2004)** - catalog line 36; **Continuous MI / kNN**. Primary-source role checked; equation-level review where load-bearing.

4. **Kevin Beyer et al. (1999)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

5. **Mikhail Gromov (1987)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

6. **Elizaveta Levina et al. (2004)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

7. **Maximilian Nickel et al. (2018)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

8. **David J. C. MacKay et al. (2005)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

9. **Yoav Benjamini et al. (1995)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

10. **Yoav Benjamini et al. (2001)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

11. **Shuyang Gao et al. (2015)** - catalog line 36; **Continuous MI / kNN**. Primary-source role checked; equation-level review where load-bearing.

12. **Fernando E. Rosas et al. (2019)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

13. **David A. Ehrlich et al. (2024)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

14. **Robert M. Gray et al. (1998)** - catalog line 36; **Method-supporting source**. Role/provenance checked; no universal re-derivation claimed.

15. **Paul L. Williams et al. (2010)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

16. **Malte Harder et al. (2013)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

17. **Belinda Phipson et al. (2010)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

18. **Paul Geladi et al. (1986)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

19. **Svante Wold (1978)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

20. **Hans R. Künsch (1989)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

21. **Weihao Gao et al. (2018)** - catalog line 36; **Continuous MI / kNN**. Primary-source role checked; equation-level review where load-bearing.

22. **Moses Charikar et al. (2002)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

23. **Kilian Weinberger et al. (2009)** - catalog line 36; **Geometry / dimension / nearest neighbours**. Role/provenance checked; no universal re-derivation claimed.

24. **Karl Pearson (1901)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

25. **D. R. Cox (1958)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

26. **John A. Nelder et al. (1972)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

27. **Saskia le Cessie et al. (1992)** - catalog line 36; **Preprocessing / regression**. Role/provenance checked; no universal re-derivation claimed.

28. **Claude E. Shannon (1948)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

29. **Aaron J. Gutknecht et al. (2025)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

30. **Johannes Rauh et al. (2014)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

31. **Abdullah Makkeh et al. (2021)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

32. **Aaron J. Gutknecht et al. (2021)** - catalog line 36; **Method-supporting source**. Role/provenance checked; no universal re-derivation claimed.

33. **Abzinger and contributors (2023)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

34. **Patricia Wollstadt and contributors (2026)** - catalog line 36; **External reference implementation**. Role/provenance checked; no universal re-derivation claimed.

35. **Aobo Lyu et al. (2026)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

36. **David Alexander Ehrlich and contributors (2023)** - catalog line 36; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

37. **Wassily Hoeffding (1963)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

38. **Svante Janson (2004)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

39. **Christos Pelekis et al. (2017)** - catalog line 36; **Inference / resampling / concentration**. Role/provenance checked; no universal re-derivation claimed.

40. **Tsachy Weissman et al. (2003)** - catalog line 36; **Method-supporting source**. Role/provenance checked; no universal re-derivation claimed.

41. **Thomas M. Cover et al. (2006)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

42. **Adam B. Barrett (2015)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

43. **Koenraad M. R. Audenaert (2007)** - catalog line 36; **Information theory**. Role/provenance checked; no universal re-derivation claimed.

44. **Chiara Barà et al. (2025)** - catalog line 113; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

45. **Kyle Schick-Poland et al. (2021)** - catalog line 113; **PID and decomposition**. Primary-source role checked; equation-level review where load-bearing.

18 Appendix D - Key repository artifacts reviewed

- Current README at exact head
- Current `METHODS.md` at exact head
- Current mathematical workflow at exact head
- Current foundational SxPID audit
- Current formal-tool adoption audit
- Current finite-alphabet convergence note
- Current support-change continuity note
- Current dependency-colored concentration note
- Current core source tree
- Current test tree
- Current formal tree
- Current output PDF tree
- v0.9.0 tag tree

19 Appendix E - Audit limitations and confidence boundaries

1. The repository's exact current archive could not be cloned because the execution environment had no outbound DNS. The GitHub-rendered tree and retrieved file bodies were used instead.
2. The committed repository PDFs could not be fetched as renderable PDF media through the available transport. Their source, inventory, and visible rendering receipt were reviewed; their binary page appearance was not independently certified here.
3. The Ehrlich authors' GitLab reference implementation was inaccessible, so statements about its exact corrective wiring remain unverified in this audit.
4. Peripheral workflow sources about AI process, Erdős problems, zeta, and social-media exchanges were classified and custody-reviewed, not mathematically re-proved. They are not load-bearing PID evidence.
5. This report identifies defects, proof gaps, and design risks. It is not a substitute for named domain-expert review, a clean-room full checkout, reproducible build, or institutional software audit.

20 Final assessment

The repository's authors have done unusually thoughtful work on semantic boundaries, failure modes, exact arithmetic, proof limitations, and source errata. The project deserves serious review. Its biggest risk is that the sheer amount and prestige of assurance machinery can obscure the simple question: **which exact claim, at which exact commit, has been reviewed by whom, across which correspondence edges?**

Fix that first. Then remove provenance-losing compatibility paths, make units and quantizer semantics structural, require joint uncertainty for continuous PID, and reduce the review protocol to a compact source-and-obligation graph. With those changes, `pid-rs` could become a model for research software that states exactly what it knows, checks, and leaves open.